



PASSO A PASSO – SPRING BOOT

Sistema de Gestão da Copa do Mundo

Guia completo: do Spring Initializr aos endpoints funcionando no Postman

1. Antes de abrir o IntelliJ

No Spring Initializr (start.spring.io), crie o projeto com Maven e Java 17 ou superior. Use as dependências abaixo:

Dependência	Para que serve
Spring Web	Permite criar endpoints REST: GET, POST, PUT, DELETE.
Spring Data JPA	Facilita a comunicação com o banco usando Repository.
H2 Database	Banco em memória usado para treino e testes.
Validation	Permite usar @NotBlank, @Positive, @Min e outras validações.
Lombok	Reduz código repetitivo com @Getter, @Setter, @NoArgsConstructor.

2. Abrindo o projeto no IntelliJ

1. Baixe o arquivo .zip gerado pelo Spring Initializr.
2. Extraia o .zip em uma pasta de fácil acesso (ex: Documentos/projetos/).
3. Abra o IntelliJ IDEA.
4. Clique em File > Open.
5. Selecione a pasta extraída do projeto — não apenas a pasta src.
6. Aguarde o Maven baixar as dependências (barra de progresso no canto inferior direito).

⚠ Se aparecer erro de SDK, vá em File > Project Structure > SDK e selecione o Java 17 instalado.

3. Crie primeiro o application.properties

Esse arquivo fica em src/main/resources. Ele configura o banco H2 e libera o console para visualizar as tabelas.

```
spring.application.name=copa

spring.datasource.url=jdbc:h2:mem:copadb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

💡 Após rodar o projeto, acesse <http://localhost:8080/h2-console> no navegador para ver as tabelas criadas pelo JPA.

4. Crie os pacotes do projeto

Dentro do pacote principal (onde está a classe com `@SpringBootApplication`), crie os pacotes abaixo. No IntelliJ: clique com o botão direito no pacote principal > New > Package.

Pacote	O que vai dentro	Por que separar?
model	Selecao, Convocacao, Partida	Representa os dados do banco (entidades JPA)
repository	SelecaoRepository, ConvocacaoRepository, PartidaRepository	Camada de acesso ao banco — só se comunica com o model
service	ConvocacaoService, PartidaService	Regras de negócio — chama o repository e processa dados
controller	SelecaoController, ConvocacaoController, PartidaController	Recebe as requisições do Postman e chama o service
dto	(Reservado para relatórios e respostas customizadas)	Objetos de transferência de dados — evita expor a entidade diretamente

💡 A ordem de criação das classes deve respeitar essa sequência: *model* → *repository* → *service* → *controller*. Cada camada depende da anterior.

5. Crie primeiro as classes Model

Comece pelo model porque as outras camadas dependem das entidades. Neste exercício, as entidades são Selecao, Convocacao e Partida.

📦 Cada classe model fica no pacote model. Adicione os imports que o IntelliJ sugerir (*Alt+Enter* sobre os erros sublinhados em vermelho).

5.1 Model Selecao

```
import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.*;
```

```

@Entity
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
public class Selecao {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Nome é obrigatório")
    private String nome;

    @NotBlank(message = "Grupo é obrigatório")
    private String grupo;

    private String fase;

    @Min(value = 0, message = "Gols marcados não pode ser negativo")
    private Integer golsMarcados;

    @Min(value = 0, message = "Gols sofridos não pode ser negativo")
    private Integer golsSofridos;

    @Positive(message = "Valor do plantel deve ser maior que zero")
    private Double valorPlantel;

    private Integer jogadoresDisponiveis = 0;
}

```

💡 *jogadoresDisponiveis representa o saldo atual de jogadores. Ele aumenta quando há convocação e diminui quando jogadores são escalados para uma partida.*

5.2 Model Convocacao

```

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.*;
import java.time.LocalDate;

@Entity
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
public class Convocacao {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private LocalDate dataConvocacao;

    @Positive(message = "Quantidade deve ser maior que zero")
    private Integer quantidade;
}

```

```

    @ManyToOne
    private Selecao selecao;
}

```

5.3 Model Partida

```

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.*;
import java.time.LocalDate;

@Entity
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
public class Partida {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private LocalDate dataPartida;

    @Positive(message = "Quantidade deve ser maior que zero")
    private Integer quantidade;

    @ManyToOne
    private Selecao selecao;
}

```

6. Depois crie os Repositories

O Repository é a camada que conversa com o banco. Ele deve ser criado depois do model. São interfaces — não classes — e o Spring cria a implementação automaticamente.

```

import org.springframework.data.jpa.repository.JpaRepository;

public interface SelecaoRepository extends JpaRepository<Selecao, Long> {
}

```

```

public interface ConvocacaoRepository extends JpaRepository<Convocacao, Long> {
}

```

```

import java.util.List;

public interface PartidaRepository extends JpaRepository<Partida, Long> {
    List<Partida> findAllByOrderByDataPartidaDesc();
}

```

💡 O método `findAllByOrderByDataPartidaDesc()` é criado automaticamente pelo Spring Data JPA com base no nome. Não precisa escrever SQL.

7. Crie os Services com as regras de negócio

O Service é onde ficam as regras: convocar, escalar, validar saldo. Ele recebe o Repository via injeção de dependência com `@RequiredArgsConstructor`.

7.1 ConvocacaoService

```
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class ConvocacaoService {

    private final ConvocacaoRepository convocacaoRepository;
    private final SelecaoRepository selecaoRepository;

    public Convocacao registrar(Convocacao convocacao) {
        Selecao selecao =
        selecaoRepository.findById(convocacao.getSelecao().getId())
            .orElseThrow(() -> new RuntimeException("Seleção não
        encontrada"));

        Integer saldoAtual = selecao.getJogadoresDisponiveis();
        selecao.setJogadoresDisponiveis(saldoAtual +
        convocacao.getQuantidade());

        selecaoRepository.save(selecao);
        convocacao.setSelecao(selecao);
        return convocacaoRepository.save(convocacao);
    }
}
```

7.2 PartidaService

```
@Service
@RequiredArgsConstructor
public class PartidaService {

    private final PartidaRepository partidaRepository;
    private final SelecaoRepository selecaoRepository;

    public Partida registrar(Partida partida) {
        Selecao selecao =
        selecaoRepository.findById(partida.getSelecao().getId())
            .orElseThrow(() -> new RuntimeException("Seleção não
        encontrada"));

        if (selecao.getJogadoresDisponiveis() < partida.getQuantidade()) {
            throw new RuntimeException("Quantidade indisponível de jogadores");
        }
    }
}
```

```

        selecao.setJogadoresDisponiveis (
            selecao.getJogadoresDisponiveis() - partida.getQuantidade()
        );

        selecaoRepository.save(selecao);
        partida.setSelecao(selecao);
        return partidaRepository.save(partida);
    }
}

```

⚠ Se o Postman retornar erro 500 nas validações de saldo, isso é esperado. O `RuntimeException` gera status 500 por padrão. O sistema está correto — o banco não foi alterado.

8. Crie os Controllers por último

O Controller recebe as requisições do Postman e chama o Service (ou Repository diretamente quando não há regra de negócio).

💡 *SelecaoController usa o SelecaoRepository diretamente porque o cadastro de seleção não tem regra de negócio — apenas salva no banco.*

8.1 SelecaoController

```

import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;
import java.util.List;

@RestController
@RequestMapping("/selecoes")
@RequiredArgsConstructor
public class SelecaoController {

    private final SelecaoRepository selecaoRepository;

    @GetMapping
    public List<Selecao> listar() {
        return selecaoRepository.findAll();
    }

    @PostMapping
    public ResponseEntity<Selecao> cadastrar(@Valid @RequestBody Selecao selecao) {
        return ResponseEntity.status(201).body(selecaoRepository.save(selecao));
    }
}

```

8.2 ConvocacaoController

```
@RestController
@RequestMapping("/convocacoes")
@RequiredArgsConstructor
public class ConvocacaoController {

    private final ConvocacaoService convocacaoService;

    @PostMapping
    public ResponseEntity<Convocacao> registrar(@Valid @RequestBody Convocacao convocacao) {
        return
        ResponseEntity.status(201).body(convocacaoService.registrar(convocacao));
    }
}
```

8.3 PartidaController

```
@RestController
@RequestMapping("/partidas")
@RequiredArgsConstructor
public class PartidaController {

    private final PartidaService partidaService;
    private final PartidaRepository partidaRepository;

    @GetMapping
    public List<Partida> listar() {
        return partidaRepository.findAllByOrderByDataPartidaDesc();
    }

    @PostMapping
    public ResponseEntity<Partida> registrar(@Valid @RequestBody Partida partida) {
        return
        ResponseEntity.status(201).body(partidaService.registrar(partida));
    }
}
```

9. Testes iniciais no Postman/Insomnia

A ordem dos testes é importante. Primeiro cadastre a seleção, depois registre a convocação e só depois registre a partida.

POST /selecoes — Cadastrar seleção

```
{
  "nome": "Brasil",
  "grupo": "A",
  "fase": "Grupos",
  "golsMarcados": 0,
  "golsSofridos": 0,
```

```
{
  "valorPlantel": 250.5
}
```

POST /convocacoes — Registrar convocação

```
{
  "dataConvocacao": "2026-05-01",
  "quantidade": 23,
  "selecao": { "id": 1 }
}
```

POST /partidas — Registrar escalação

```
{
  "dataPartida": "2026-06-12",
  "quantidade": 11,
  "selecao": { "id": 1 }
}
```

GET /selecoes — Verificar saldo

Verifique se o campo jogadoresDisponiveis ficou com valor 12 (23 convocados – 11 escalados = 12).

10. Relatórios — implementação completa

Todos os relatórios são endpoints GET que apenas consultam dados — não alteram o banco. Eles são adicionados nos Controllers e Repositories existentes.

Relatório	Endpoint	Onde implementar
Por grupo	GET /selecoes/grupo	SelecaoController + SelecaoRepository
Limites	GET /selecoes/limites	SelecaoController + SelecaoRepository
Histórico	GET /partidas/historico?dataInicial=&dataFinal=	PartidaController + PartidaRepository
Mais escaladas	GET /partidas/mais-escaladas?dataInicial=&dataFinal=	PartidaController + PartidaRepository

10.1 Relatório: seleções por grupo

Adicione no SelecaoRepository:

```
import org.springframework.data.jpa.repository.Query;

@Query("SELECT s FROM Selecao s ORDER BY s.grupo ASC, s.nome ASC")
```



```
List<Selecao> findAllOrderByGrupo();
```

Adicione no SelecaoController:

```
@GetMapping("/grupo")
public List<Selecao> listarPorGrupo() {
    return selecaoRepository.findAllOrderByGrupo();
}
```

10.2 Relatório: seleções nos limites (0 ou 100 jogadores)

Adicione no SelecaoRepository:

```
List<Selecao>
findByJogadoresDisponiveisLessThanEqualOrJogadoresDisponiveisGreaterThanEqual(
    Integer min, Integer max);
```

Adicione no SelecaoController:

```
@GetMapping("/limites")
public List<Selecao> listarNosLimites() {
    return selecaoRepository
        .findByJogadoresDisponiveisLessThanEqualOrJogadoresDisponiveisGreaterThanEqual(0, 100);
}
```

10.3 Relatório: histórico por período

Adicione no PartidaRepository:

```
import java.time.LocalDate;

List<Partida> findByDataPartidaBetweenOrderByDataPartidaAsc(
    LocalDate dataInicial, LocalDate dataFinal);
```

Adicione no PartidaController:

```
import java.time.LocalDate;
import org.springframework.format.annotation.DateTimeFormat;

@GetMapping("/historico")
public List<Partida> historico(
    @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate dataInicial,
    @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate dataFinal) {
    return partidaRepository
        .findByDataPartidaBetweenOrderByDataPartidaAsc(dataInicial, dataFinal);
}
```

💡 No Postman, chame assim: **GET**
/partidas/historico?dataInicial=2026-06-01&dataFinal=2026-06-30

10.4 Relatório: mais escaladas por período

Adicione no PartidaRepository:

```
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

@Query("SELECT p FROM Partida p " +
      "WHERE p.dataPartida BETWEEN :dataInicial AND :dataFinal " +
      "ORDER BY p.quantidade DESC")
List<Partida> findMaisEscaladasNoPeriodo(
    @Param("dataInicial") LocalDate dataInicial,
    @Param("dataFinal") LocalDate dataFinal);
```

Adicione no PartidaController:

```
@GetMapping("/mais-escaladas")
public List<Partida> maisEscaladas(
    @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate
dataInicial,
    @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate
dataFinal) {
    return partidaRepository.findMaisEscaladasNoPeriodo(dataInicial, dataFinal);
}
```

11. Frase simples para lembrar a ordem

Model → Repository → Service → Controller

- Model primeiro — representa os dados. Sem ele, nada existe.
- Repository depois — depende do model para saber qual entidade acessar.
- Service depois — usa o repository e aplica a regra de negócio.
- Controller por último — expõe a API para o Postman chamar.

12. Checklist para o aluno

Use este checklist antes de entregar. Cada item precisa estar funcionando.

- ☐ O projeto abriu no IntelliJ sem erro no Maven.
- ☐ O application.properties foi configurado corretamente.
- ☐ Os pacotes model, repository, service, controller e dto foram criados.
- ☐ As três entidades foram criadas com @Entity e os imports corretos.
- ☐ Convocacao e Partida possuem @ManyToOne com Selecao.
- ☐ Os repositories foram criados com JpaRepository.
- ☐ A convocação aumenta o jogadoresDisponiveis da seleção.
- ☐ A partida diminui o jogadoresDisponiveis e valida o saldo antes.
- ☐ Os endpoints foram testados no Postman na ordem correta.
- ☐ O H2 Console mostra as tabelas SELECAO, CONVOCACAO e PARTIDA criadas.
- ☐ Os 4 relatórios (GET) retornam dados corretamente no Postman.